

GPU Pipeline for Accelerated Search of an Upper Bound on the Matrix Multiplication Exponent

Minh Phuc Nguyen

Department of Computer Science

Union College

Schenectady, New York, USA

nguyenp@union.edu

Abstract—The search for Simplifiable Strong Uniquely Solvable Puzzles (SUSPs) is one promising way to improve the upper bound of the matrix multiplication exponent, but its core simplification algorithm is computationally expensive. Existing heterogeneous implementations suffer from severe GPU underutilization and host synchronization bottlenecks, resulting in poor end-to-end throughput. We propose a solution using batched GPU simplification together with an asynchronous producer-consumer pipeline. Our approach achieves up to a $5.75\times$ end-to-end throughput speedup over the original implementation by increasing GPU active time from 9–22% to nearly 100%, and kernel SM throughput from 13–23% to 28–37%. Applying this pipeline enabled the discovery of a (79, 10)-SUSP, which surpasses the previous (78, 10)-SUSP maximum found by the baseline. These results show that maximizing the overlap between CPU and GPU is the key to applying high-performance computing to complex research problems.

Index Terms—matrix multiplication, strong uniquely solvable puzzle, GPU computing, CUDA, combinatorial search, heterogeneous computing

I. INTRODUCTION

Anderson and Le introduced an algorithm for searching for Simplifiable Strong Uniquely Solvable Puzzles (Simplifiable SUSP), a combinatorial object used in the Cohn-Umans framework that can imply an upper bound on the exponent ω , which defines the $O(n^\omega)$ complexity of matrix multiplication algorithms [1], [2], [3]. The goal of the program is to search for Simplifiable SUSP, which occur at a very small rate within a search space whose naive ordered-matrix representation contains $3^{s \times k}$ configurations. Each candidate is called a puzzle. An (s, k) -puzzle P is defined as $P \subseteq \{1, 2, 3\}^k$ with $|P| = s$. For fixed width k , the larger the size-to-width ratio of a Simplifiable SUSP, the tighter the upper bound on ω ; therefore, the goal is to search for Simplifiable SUSP with large sizes, such as (35, 8) or (52, 9), as in the study by Anderson and Le [3].

To search efficiently, they developed an Iterated Local Search (ILS) algorithm that moves through a state space in which each state is a candidate puzzle, and nearby states are obtained through small modifications of the current puzzle [4], [3]. The core of the program is the SIMPLIFY algorithm, which is used to compute a heuristic score for ranking each candidate and determining which regions of the state space should be explored next [3].

However, their search algorithm has not yet achieved optimal utilization of the available computing hardware. The SIMPLIFY algorithm has a worst-case time complexity of $O(s^{5.5})$. Although computationally expensive, it contains abundant fine-grained parallel work. Therefore, a feasible approach is to utilize CUDA and GPU computing to accelerate simplification during the search process [5]. Previous work achieved a $10\times$ speedup for the isolated SIMPLIFY step on a GPU compared with a single CPU core [3]. However, this acceleration for the isolated SIMPLIFY step has not been reflected in the actual search program.

In the search program, throughput is defined as the number of puzzles that the program can simplify in 1 second. In our experiment, Anderson and Le’s implementation with the GPU doing per-puzzle simplification alters end-to-end search throughput by only $0.96\times$ to $1.02\times$ compared to using only CPU to both search and simplify. This is due to the overhead of per-puzzle data allocation, constant PCIe transfers between CPU and GPU, and frequent host-device synchronization. Because the kernel execution time for a single candidate is short, PCIe transfer latency and CUDA driver dispatch overhead completely dominate the per-puzzle wall-clock time.

Based on these current problems, we redesigned the SIMPLIFY algorithm so that it can run in batches on the GPU. Hundreds of same-shape puzzles are packed to simplify concurrently, producing results that exactly match the CPU baseline. Compared to the original per-puzzle GPU implementation, running in batches of 256 reduces simplification time for each puzzle by $26.34\times$ to $117.69\times$. This relative speedup is greater for larger SUSP sizes, because their inherent computational workload saturates the GPU execution units more thoroughly when processed in batches. Additionally, we redesigned the pipeline for the search program to overlap the search process on the CPU and batched simplification on the GPU. This increases GPU active time from 9–22% to 99–100%. Across various benchmark targets, our batched pipeline improves end-to-end simplification throughput by $3.09\times$ – $5.75\times$ over the original Anderson and Le search implementation with per-puzzle Simplify on GPU.

This paper has 3 main contributions:

- 1) We design a batched CUDA kernel that simplifies hundreds of same-shape puzzles concurrently.

- 2) We implement a persistent heterogeneous search pipeline; by keeping device memory allocations alive and minimizing host-device synchronization, this pipeline overlaps CPU-side candidate generation with GPU simplification.
- 3) We evaluate the system-level conditions under which batched acceleration becomes effective, showing that our pipeline achieves an end-to-end throughput speedup of $3.09\times$ to $5.75\times$ over the original search implementation. This speedup enabled the discovery of a new (79, 10)-SUSP, surpassing the baseline’s previous record.

This work is a case study of heterogeneous CPU-GPU pipeline design. We adapt several workflow optimization ideas from the machine learning field: batch simplification is inspired by mini-batch gradient descent [6], and the producer-consumer search pipeline is inspired by pipeline parallelism [7]. We believe that these fundamental ideas for maximizing GPU utilization can be broadly applied to overcome critical host-device bottlenecks in other high-performance computing problems.

II. BACKGROUND AND BOTTLENECK

This section introduces the two core algorithmic components of the Anderson and Le’s search program: the SIMPLIFY operation and the Iterated Local Search loop. We also identify the performance bottleneck that motivates our work.

A. Simplify Operation

SIMPLIFY checks whether a candidate puzzle is a simplifiable SUSP by iteratively projecting its 3D edge-graph onto three 2D planes and removing edges that doesn’t satisfy requirements [3]. In the worst case, the algorithm has a time complexity of $O(s^{5.5})$, but it contains abundant fine-grained parallel work that makes GPU acceleration attractive.

B. Iterated Local Search Algorithm

The search repeatedly expands promising candidates and explores their neighbors. The original Anderson and Le’s search program uses OpenMP to exploit multiple CPU cores, with CPU threads acting as workers that can expand the search from different frontier states concurrently. Some steps in the search generate many candidates with the same shape. These candidates are good inputs for batched GPU simplification.

C. Bottleneck in the Heterogeneous Search

Figure 1 shows the original staged searcher with and without its per-puzzle CUDA path. The GPU-enabled path changes search evaluation throughput by only $0.96\times$ – $1.03\times$. This is the central motivation for the paper: isolated GPU kernels are not enough when host-device synchronization and per-puzzle control remain in the inner search loop.

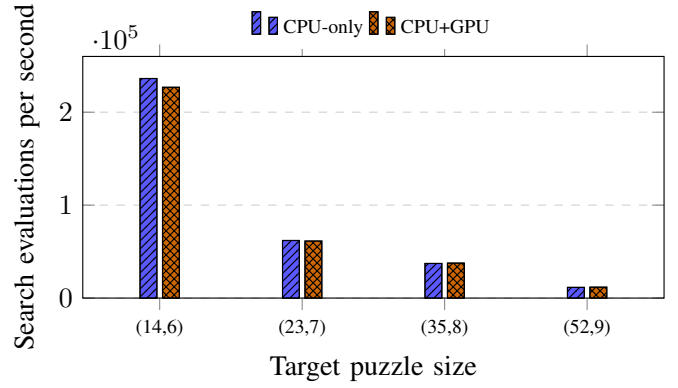


Fig. 1. End-to-end search evaluation throughput of the original staged searcher. Utilizing the GPU for per-puzzle simplification provides no overall speedup, as the host remains a bottleneck within the control loop.

III. BATCHED GPU SIMPLIFICATION

In this section, we present two main changes to optimize the SIMPLIFY algorithm using CUDA compared with the baseline implementation based on Anderson and Le, as summarized in Algorithm 1. First, we use persistent host-pinned and device memory throughout the entire search process and organize the data to maximize coalesced memory access. Second, the per-candidate convergence state is maintained on the GPU to prevent redundant work. The correctness and performance of the batched GPU implementation are evaluated in Section V.

A. Persistent Batched Memory Layout

The fundamental change in our design is the transition from simplifying individual puzzles to batches of puzzles with the same dimensions. In the original approach, each generated puzzle was stored separately in pageable host memory. Transferring each puzzle to the GPU required separate memory-copy and kernel-launch operations, resulting in severe GPU underutilization. Table I shows that processing puzzles individually cannot fully utilize the Streaming Multiprocessors (SMs), achieving only $\sim 13 - 23\%$ utilization across tested sizes, whereas our batched approach increases this value to $\sim 27 - 37\%$.

Algorithm 1 Baseline per-puzzle simplification [3]

Require: A candidate puzzle P

Ensure: Heuristic score: remaining edge count for P

- 1: Allocate GPU memory for P and its 3D graph
- 2: Copy P from CPU memory to GPU memory
- 3: Build the initial 3D graph for P on the GPU
- 4: $consecutiveNoRemovalPlanes \leftarrow 0$
- 5: **while** $consecutiveNoRemovalPlanes < 3$ **do**
- 6: Launch projection and edge-removal kernels for the next 2D plane
- 7: Delete corresponding edges from the 3D graph on the GPU
- 8: Copy the edge-removal flag to the CPU and synchronize
- 9: **if** some edge was removed from the 3D graph **then**
- 10: $consecutiveNoRemovalPlanes \leftarrow 0$
- 11: **else**
- 12: Increment $consecutiveNoRemovalPlanes$ by 1
- 13: Compute partial counts of remaining edges on the GPU
- 14: Copy partial counts to the CPU
- 15: Free per-puzzle CPU and GPU memory

TABLE I
AVERAGE SM UTILIZATION FOR SIMPLIFY KERNELS

Approach	14x6	23x7	35x8	52x9
Per-Puzzle (Original)	13.1%	14.2%	18.1%	23.0%
Batched (Proposed)	27.8%	29.2%	36.0%	37.3%

Note: Profiling was performed on an NVIDIA RTX 5070 Ti using Nsight Compute (ncu) over 10 iterations of the core simplification loop. We report the `sm_throughput.avg.pct_of_peak_sustained_active` metric, representing the average percentage of peak sustained active cycles across all SMs for the dominant compute kernels.

To support batching and efficient memory access, we introduce a packed candidate representation. Because each puzzle cell can take only one of three values, each cell is encoded using two bits. For the puzzle dimensions examined in this study, an entire puzzle row fits within a single 64-bit word. Candidate puzzles with the same dimensions (s, k) are stored in a single contiguous batch buffer. By packing rows into 64-bit integers, the kernel reduces memory traffic and enables compact integer operations (e.g., shifts, bitwise AND/OR, and population counts) to evaluate edge validities. Organizing the data as a one-dimensional array makes offset calculations fully predictable and allows GPU threads to access memory in a coalesced manner. The packed representation also reduces the amount of data transferred from the host to the device.

We further use persistent CUDA resources to remove memory-allocation overhead entirely from the core search loop. As shown in Algorithm 1, the previous method allocates GPU memory at line 1 and releases it at line 15 for every individual puzzle, introducing latency into the main loop. To address this problem, our batch buffers on both the host and device are allocated only once at the beginning of the search. On the host side, we use pinned memory. This is essential because it enables asynchronous data transfers through `cudaMemcpyAsync` and avoids implicit synchronization points that would stall the CPU search process [8]. The GPU evaluator retains these persistent memory regions throughout the entire search, while the strict queue-control mechanisms discussed in Section IV ensure that active batches are not overwritten in the main buffer.

B. Device-Resident Convergence State

One challenge of batched SIMPLIFY is that different puzzles within the same batch may need different numbers of iterations to converge. Continuing to run the GPU kernels after all puzzles in the batch have already converged wastes computational resources.

To eliminate this bottleneck, we propose a persistent GPU-side schedule, as illustrated in Algorithm 2. The CPU side will keep launching the projection and edge-removal kernels up to the theoretical maximum bound. After each iteration, the CPU performs a lightweight asynchronous copy of the number of active puzzles to the host. If the counter reaches zero, the CPU loop breaks early.

To avoid redundant computations for puzzles that converge early, the convergence state is managed directly in the device

Algorithm 2 Proposed batched GPU simplification

Require: A batch $\mathcal{B}$ of N puzzles of size $s \times k$; $T_{\max} = 3(s^3 - s)$ [3]
Ensure: Heuristic scores (remaining edge counts) for all puzzles in $\mathcal{B}$

- 1: Async-copy $\mathcal{B}$ from pinned host buffer to persistent device buffer
- 2: Initialize per-puzzle convergence flags on device (all active)
- 3: **for** $iter = 1$ **to** $T_{\max}$ **do**
- 4: Launch projection and edge-removal kernels for the next 2D plane
 (each thread checks its puzzle's convergence flag and returns immediately if already converged)
- 5: Delete corresponding edges from the 3D graph on the GPU
- 6: Async-copy active-puzzle counter from device to host
- 7: Synchronize CUDA stream
- 8: **if** active-puzzle counter = 0 **then**
- 9: **break**
- 10: Launch reduction kernel; compute per-puzzle edge counts on device
- 11: Async-copy results from device to pinned host buffer

memory. At the beginning of each kernel execution, the GPU threads read the status flag of its puzzle. If the puzzle has already converged in a previous iteration, that thread will immediately exit early, skipping remaining computational logic in the kernel. Because we use configuration that every thread in a block works on the same puzzle, there will be less thread divergence related to the early exiting.

IV. INTEGRATION INTO STAGED SEARCH

This section will show the integration of this batched GPU Simplify with the Iterated Local Search (ILS) algorithm of Anderson and Le [3]. First, we show the search state transformations that generate the dense candidate sets required for batch processing. Second, we present an asynchronous Producer-Consumer system to efficiently coordinate the OpenMP CPU search threads with the GPU pipeline. The performance evaluation of the complete system is presented in Section V.

A. Candidate Generation for Batched Processing

In the Iterated Local Search (ILS) program of Anderson and Le [3], neighborhood exploration is performed through four main transformations, which we divide into two categories: *Sparse* and *Dense*. Single-cell mutations and element permutations belong to the *Sparse* group, as they only generate a small set of candidates with a quantity that scales as $O(sk)$. Conversely, brute-force last-row generation and active row replacement belong to the *Dense* group, producing an exponential volume of 3^k candidates per transformation.

Among these, *Dense workloads* are the target we aim for with batched SIMPLIFY. For puzzles with a width of $k \geq 6$, just a single *dense* transformation generates hundreds of candidates, which is more than enough to immediately saturate multiple fixed-size GPU batches (e.g., 256 candidates). As for the *Sparse workloads*, we leave them to be processed by the CPU to avoid transferring small, fragmented data.

In Anderson and Le's previous model [3], CPU thread workers encountering a dense workload transformation would loop through each candidate and evaluate SIMPLIFY individually as normal. However, in the enhanced program, we pack each batch of candidates into a single object called a Task. A Task contains the raw data of all the candidates, along with

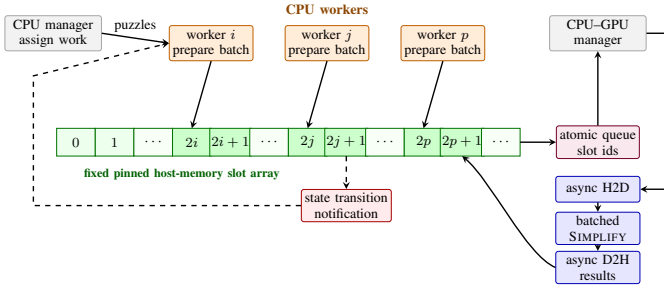


Fig. 2. Producer-consumer workflow for batched GPU simplification. CPU workers enqueue task metadata from pinned slots, while a dedicated GPU orchestration thread manages execution and state updates.

metadata, such as the Task ID, the thread that created the task, and the actual number of candidates in the task (since transformations do not always generate a number of candidates perfectly divisible by the batch capacity).

B. Asynchronous Producer-Consumer Workflow

In an OpenMP environment, all threads independently launching kernels and managing data transfers creates a performance bottleneck. Concurrent kernel launches from multiple host threads introduce implicit synchronization points within the CUDA driver [9], forcing CPU workers to idle.

To maintain thread independence, we implemented double-buffering for each CPU worker. Each thread is exclusively allocated two memory slots (e.g., thread i receives slots $2i$ and $2i+1$). This configuration provides sufficient depth to allow a worker to assemble its next candidate batch while its previous batch is concurrently transferring and executing on the GPU. A slot is a buffer large enough to hold an entire batch candidate. During initialization, all slots are allocated contiguously via a single `cudaMallocHost` call, creating a shared pinned memory region [9], [8]. This eliminates dynamic allocation overhead during execution.

Workers interact only with their two designated slots. When a worker finishes a dense transformation and aggregates a batch, it encapsulates the data into a Task and loads it into an available slot. Each worker tracks the state of its own slots locally. If both slots are full, the worker thread blocks until a slot becomes empty, providing natural back-pressure.

After loading a Task into a slot, the worker enqueues only the Task metadata into a global shared queue, avoiding the cost of copying raw data. This queue uses atomic operations to provide thread-safe concurrent enqueueing and to maintain processing order among producers.

To separate GPU execution from CPU search logic, we use a dedicated GPU manager thread. This thread does not generate candidates; it only consumes Tasks from the global queue. A static device memory buffer is allocated once on the GPU and managed exclusively by this thread. Its workflow is simple: read metadata to locate the host slot, transfer data to the device buffer, execute the bounded simplification schedule, and copy results back to the host slot, as illustrated in Figure 2.

TABLE II
EXPERIMENTAL ENVIRONMENT AND DEFAULT MEASUREMENT SETTINGS.

Component	Configuration
CPU	AMD Ryzen 9 9900X, 12 cores / 24 threads
GPU	NVIDIA GeForce RTX 5070 Ti, 16 GiB, device 0
RAM	64 GiB
OS	Ubuntu 24.04.4 LTS
CUDA	13.2
CPU baseline	OpenMP, 24 threads
GPU batch size	256 candidates unless noted
Measurement	wall-clock timers; mean of 10 independent runs

To notify producers when a Task finishes, the manager thread uses `std::condition_variable` combined with `std::atomic` flags [10].

V. EVALUATION

We evaluate our batched GPU simplification pipeline across four puzzle dimensions that represent the largest known SUSPs for each actively researched width k . We assess correctness, isolated simplification throughput, kernel-level GPU utilization, and end-to-end staged search performance, comparing our implementation against the original per-puzzle heterogeneous baseline.

A. Experimental Setup

All experiments were conducted using the hardware and software environment detailed in Table II.

To evaluate the performance of our batched GPU simplification and staged search, we benchmarked the system across four specific puzzle dimensions: (14, 6)-puzzles, (23, 7)-puzzles, (35, 8)-puzzles, and (52, 9)-puzzles. These dimensions were chosen because they represent the largest SUSPs found by Anderson and Le for each actively researched width k . Measuring performance from these specific instances allows us to address the primary bottleneck in finding better SUSPs, a step that directly contributes to improving the upper bound of ω .

To save time and focus on the throughput and performance of the search program, rather than focusing on finding new SUSPs, we start the search process with the help of a seed from an already known SUSP to begin searching in the most frontier search size.

For end-to-end throughput measurements, we recorded the number of search evaluations per second over a 30-second steady-state execution window. GPU resource profiling, such as Streaming Multiprocessor (SM) utilization, was captured using `nvtop` over a 15-second tracking window, while detailed kernel-level metrics were analyzed using NVIDIA Nsight Compute (`ncu`).

B. Correctness Validation

To ensure that GPU acceleration strictly preserves the exactness of puzzle simplification, we integrated an automated testing pipeline into the project’s Google Test suite. For each dimension, we generated a batch of 256 randomized initial puzzles and simplified them using three distinct execution paths: the unaccelerated CPU sequential baseline, the legacy

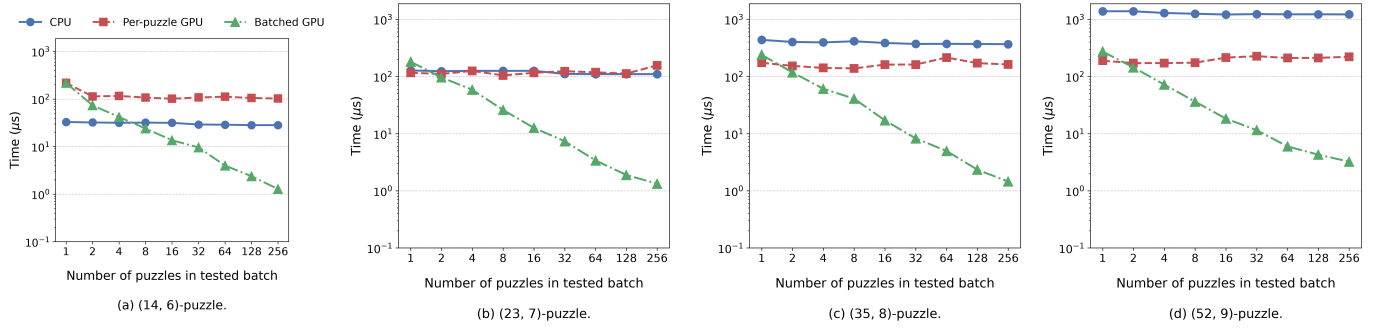


Fig. 3. Average simplification time per puzzle vs. batch size for four target dimensions. The batched GPU evaluator outperforms both the original GPU offload and the CPU baseline at larger batch sizes. Note: The CPU baseline represents a sequential execution of the batch using a single OpenMP thread.

per-puzzle GPU approach, and our proposed batched GPU architecture. We executed this validation suite 10 separate times.

Across all evaluated dimensions and randomized test inputs, we asserted strict element-wise equality on the resulting 3D graphs, the internal representation of remaining active edges. Both the per-puzzle and batched GPU logic produced bit-for-bit identical results compared to the CPU ground truth with zero mismatches, confirming the correctness of our algorithms.

C. Isolated Simplification Benchmark

We benchmarked three distinct execution paths: the sequential CPU OpenMP baseline, the original per-puzzle CUDA offload, and our enhanced batched CUDA simplification. We systematically varied the batch size from 1 to 256 (in powers of two) and measured the total execution time for the batch. To derive a normalized metric for comparison, we report the average simplification time required per individual puzzle.

The results of this experiment are illustrated in Figure 3. First, at a batch size of 1, the batched GPU pipeline can exhibit higher overhead compared to sequential CPU execution or the original GPU offload. This overhead comes from kernel launching and synchronization. However, as the batch size increases, this overhead is splitted across multiple candidate puzzles. At a batch size of 256, the batched CUDA simplification reduces the average processing time per puzzle, outperforming both the CPU baseline and the original per-puzzle GPU approach. Because the host-side loop breaks early upon convergence, it avoids launching the theoretically maximum number of kernels, which explains the microsecond-level execution times shown in Figure 3.

D. Kernel-Level Architectural Profiling

To understand the hardware execution dynamics of the batched simplification pipeline, we profiled the isolated execution of the (52,9)-puzzles puzzle dimension (using a batch size of 256 for our enhanced architecture) using NVIDIA Nsight Compute (`ncu`) and compared it against the legacy per-puzzle GPU approach. Note that this section reports SM pipeline saturation (a finer-grained Nsight Compute metric measuring execution pipeline occupancy), which is distinct from the `sm_throughput.avg.pct_of_peak`

`_sustained_active` metric reported in Table I; both measure different aspects of SM utilization. The architectural analysis reveals that the legacy kernels suffer from severe grid under-sizing, achieving a mere 0.04% SM pipeline saturation and 4.71% memory bandwidth utilization. A single puzzle simply lacks sufficient parallel work to populate the GPU.

By contrast, our batched architecture successfully saturates the device. The core batched edge evaluation kernel is predominantly compute-bound, achieving an SM pipeline saturation of 76.11% compared to a memory bandwidth utilization of 43.59%. However, the subsequent 2D graph projection kernel suffers from latency issues, yielding 28.73% SM throughput and 28.86% memory throughput. This performance drop is attributed to the irregular, uncoalesced memory access patterns inherent in projecting and pruning the 3D puzzle graph.

Consequently, while the asynchronous producer-consumer pipeline successfully maintains near 100% device residency (Table III), the average structural SM pipeline saturation across the batched simplification process settles at approximately 37% (Table I). This analysis shows that memory latency and irregular memory access patterns become the main bottlenecks of the current algorithm.

E. End-to-End Staged Search Throughput

To evaluate the end-to-end performance impact of our batched GPU simplification pipeline, we integrated it into the full staged iterated local search loop and measured total search throughput.

We benchmarked the search program on the same four target dimensions, comparing the original staged search (Baseline Search), which uses OpenMP CPU and per-puzzle GPU simplification, with our batched GPU-assisted search (Enhanced Search). Throughput is measured in total search evaluations per second.

The results, alongside the average GPU active time, are presented in Table III. The data show that the Enhanced Search achieves substantial end-to-end speedups ranging from $3.09\times$ to $5.75\times$ over the Baseline Search. As the problem dimension grows, the speedup has tendency to increase.

While the Baseline Search leaves the GPU heavily under-utilized (9%–22% active time) due to the host remaining in the simplification control loop, the Enhanced Search maintains

TABLE III
END-TO-END SEARCH THROUGHPUT AND GPU UTILIZATION

Size	Approach	GPU Active	Evals/sec	Speedup
14x6	Baseline Search	22%	226,799	-
	Enhanced Search	99%	699,680	3.09x
23x7	Baseline Search	11%	61,339	-
	Enhanced Search	100%	265,081	4.32x
35x8	Baseline Search	11%	37,603	-
	Enhanced Search	100%	121,955	3.24x
52x9	Baseline Search	9%	11,712	-
	Enhanced Search	100%	67,325	5.75x

Note: The Baseline Search uses the original legacy per-puzzle GPU offload path combined with a 24-thread OpenMP CPU search. For measurement details, see Section V-A. We report the *GPU Active Time* (device residency), which indicates the percentage of time one or more kernels were executing on the GPU. This metric demonstrates pipeline occupancy rather than SM computational unit saturation (which is reported in Table I).

near-perfect 99%–100% GPU active time across all tested targets.

VI. DISCUSSION AND LIMITATIONS

We conducted the new search pipeline with batched GPU simplification over a 10-day period and discovered a new simplifiable SUSP of size (79,10)-puzzles, surpassing the previous maximum of (78,10)-puzzles found by Anderson and Le. While this discovery demonstrates the efficacy of the expanded searchable frontier, the new puzzle is not yet large enough to establish a new upper bound for the matrix multiplication exponent ω .

One possible explanation is that the candidate space grows exponentially with s and k . Therefore, even a throughput improvement requires extended execution time on large-scale clusters to potentially enable a breakthrough.

Another possibility is, for the actively studied small widths, especially $k < 10$, larger simplifiable SUSPs may not exist beyond the known examples. Our experiments do not prove such a statement. However, these results suggest that applying the enhanced throughput to narrow widths ($k \leq 10$) may yield diminishing returns. We therefore plan to move toward larger-width puzzles, especially $k \geq 11$. These searches are more difficult, but they also generate dense same-shape candidate batches, where the batched GPU simplification is most useful.

VII. RELATED WORK

GPU acceleration of combinatorial search has been studied in several contexts. Barnat et al. [11] accelerated strongly connected component computation on CUDA by keeping graph traversal host-driven while offloading bulk frontier marking to the GPU, the same CPU/GPU partition we adopt. In SAT solving, ParaFROST [12] introduced the first GPU-parallel inprocessing solver: the CDCL search remains on the CPU, while variable elimination, clause compaction, and a data-parallel garbage collector run on the GPU, achieving up to a $2\times$ speedup over sequential inprocessing. Our work applies the same principle to combinatorial object search: ILS state expansion stays on the CPU, and the expensive

SIMPLIFY scoring function is executed as a batched GPU kernel. In computational biology, GPU-BLAST [13] and G-BLASTN [14] accelerate protein and nucleotide database search, respectively, by batching multiple query sequences into a single GPU invocation. G-BLASTN further demonstrates a pipeline mode that overlaps host query preparation with device alignment, achieving up to a $14.8\times$ speedup, sharing the same producer-consumer overlap as our work. In computational drug discovery, Uni-Dock [15] docks molecules in batches across concurrent GPU threads, explicitly optimizing CPU-GPU data flow to eliminate host bottlenecks, and achieves more than a $1000\times$ speedup over single-threaded AutoDock Vina when screening tens of millions of compounds, addressing the same per-candidate scoring bottleneck as our work. Finally, Luong et al. [16] study GPU-accelerated local-search metaheuristics and propose guidelines for CPU-GPU data transfer optimization and mapping neighbor evaluations to GPU threads, reporting up to $80\times$ acceleration on large combinatorial instances. Their approach parallelizes by running many independent search instances simultaneously; our approach instead batches the inner evaluation kernel of a single ILS instance, requiring no replication of search state.

VIII. CONCLUSION

This paper shows that the main simplification bottleneck in staged simplifiable SUSP search can be moved from per-candidate evaluation to batched GPU throughput. The original per-puzzle CUDA path accelerates isolated simplification but does not improve the full search. By packing same-shape candidates, reusing persistent CUDA resources, managing convergence via an isolated orchestration thread, and integrating batches into dense candidate-generation phases, the enhanced searcher keeps the GPU better utilized during staged search.

On the evaluated targets, the resulting pipeline improves end-to-end search evaluation throughput by $3.09\times$ – $5.75\times$ over the original staged implementation. This enhanced capability enabled the discovery of a new (79,10)-puzzles, the largest known to us. While this result does not yet establish a new matrix multiplication exponent bound, it demonstrates that extreme-scale simplifiable SUSP searches are more practical on consumer-grade GPU hardware.

ACKNOWLEDGMENTS

The author thanks Professor Matthew Anderson for his direct support, guidance, and feedback throughout this project. The author also thanks Professor Anderson and Vu Le for providing the foundation for this work through their paper on efficiently-verifiable strong uniquely solvable puzzles.

In accordance with IEEE policy on the use of generative AI (IEEE PSPB Operations Manual, Section 8.2.1.B.10), the author discloses that AI language model tools were used solely to assist with writing and phrasing in the preparation of this paper. Such editorial use for grammar and language improvement is recognized by IEEE as acceptable practice and falls outside the mandatory disclosure scope; nevertheless, this disclosure is provided in the interest of transparency. All ideas,

research, experiments, implementation, and analysis presented in this paper are entirely the original work of the author.

REFERENCES

- [1] H. Cohn and C. Umans, “A group-theoretic approach to fast matrix multiplication,” in *44th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2003, pp. 438–449.
- [2] H. Cohn, R. Kleinberg, B. Szegedy, and C. Umans, “Group-theoretic algorithms for matrix multiplication,” in *46th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2005, pp. 379–388.
- [3] M. Anderson and V. Le, “Efficiently-verifiable strong uniquely solvable puzzles and matrix multiplication,” in *Computing and Combinatorics – 29th International Conference (COCOON 2023), Hawaii, HI, USA, December 15–17, 2023, Proceedings, Part II*, ser. Lecture Notes in Computer Science, W. Wu and G. Tong, Eds., vol. 14423. Springer, 2023, pp. 41–54.
- [4] H. R. Lourenço, O. C. Martin, and T. Stützle, “Iterated local search,” in *Handbook of Metaheuristics*, ser. International Series in Operations Research & Management Science, F. Glover and G. A. Kochenberger, Eds. Kluwer Academic Publishers, 2003, vol. 57, pp. 321–353.
- [5] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA,” *Queue*, vol. 6, no. 2, pp. 40–53, 2008.
- [6] L. Bottou, “Large-scale machine learning with stochastic gradient descent,” in *Proceedings of the 19th International Conference on Computational Statistics (COMPSTAT)*. Springer, 2010, pp. 177–186.
- [7] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, “GPipe: Efficient training of giant neural networks using pipeline parallelism,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [8] NVIDIA Corporation, “CUDA C++ Best Practices Guide,” 2026, version 13.3, accessed 17 June 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>
- [9] —, “CUDA C++ Programming Guide,” 2026, version 13.3, accessed 12 July 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [10] International Organization for Standardization, “ISO/IEC 14882:2020: Programming languages – C++,” 2020, international standard.
- [11] J. Barnat, P. Bauch, L. Brim, and M. Ceska, “Computing strongly connected components in parallel on CUDA,” in *2011 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 2011, pp. 544–555.
- [12] M. Osama, A. Wijs, and A. Biere, “SAT solving with GPU accelerated inprocessing,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)*, ser. Lecture Notes in Computer Science, vol. 12651. Springer, 2021, pp. 133–151.
- [13] P. D. Vouzis and N. V. Sahinidis, “GPU-BLAST: Using graphics processors to accelerate protein sequence alignment,” *Bioinformatics*, vol. 27, no. 2, pp. 182–188, 2011.
- [14] K. Zhao and X. Chu, “G-BLASTN: accelerating nucleotide alignment by the GPU,” *Bioinformatics*, vol. 30, no. 10, pp. 1384–1391, 2014.
- [15] Y. Yu, C. Cai, J. Wang, Z. Bo, Z. Zhu, and H. Zheng, “Uni-Dock: GPU-accelerated docking enables ultralarge virtual screening,” *Journal of Chemical Theory and Computation*, vol. 19, no. 11, pp. 3336–3345, 2023.
- [16] T. V. Luong, N. Melab, and E.-G. Talbi, “GPU computing for parallel local search metaheuristic algorithms,” *IEEE Transactions on Computers*, vol. 62, no. 1, pp. 173–185, 2013.